SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: *Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3 **Duration: 1 Hour**

Total Marks: 30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Roshitha - 192424400 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

i. D1 cannot work Night shift

ii. D2 must work before D3 (Morning < Afternoon < Night)

iii. Only one doctor per shift

iv. D3 cannot work Morning

v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule.

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

i. Movement allowed: Up, Down, Left, Right

ii. Cost of each move = 1

iii. Cannot pass through obstacles

iv. Use Manhattan Distance heuristic

Using the above constraints and the BFS Search Algorithm, show step-by-step node expansion and priority queue updates, and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

• The robot can move up, down, left, right

• Some cells are blocked (obstacles) and cannot be traversed

• The robot has limited sensing capability (can only observe adjacent cells)

• Each movement has a cost of 1, but entering risky zones has an additional cost of 2

• The robot must minimize total cost while ensuring safe navigation

• The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as: Intelligent agents, Rationality, Environment type.

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightag	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)

	e			
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification

1 Answer:

This is a Constraint Satisfaction Problem (CSP): the variables are the three doctors D1, D2, D3; the domain of each variable is {Morning, Afternoon, Night}; and the constraints are D1 ≠ Night, D3 ≠ Morning, all shifts distinct, and D2's shift must come before D3's shift in the order Morning < Afternoon < Night.

Instead of assigning shifts one at a time and only checking constraints afterwards, this solution prunes each doctor's domain in advance (forward checking) and always assigns the doctor with the fewest remaining legal options first (the Minimum-Remaining-Values heuristic). Because only three doctors are involved, MRV happens to pick D1 first here as well, but the domains it works with are pre-pruned rather than the full {Morning, Afternoon, Night} set.

Search trace:

- Initial domains after applying unary constraints: D1 = {Morning, Afternoon}, D2 = {Morning, Afternoon, Night}, D3 = {Afternoon, Night}.
- D1 has the fewest options (2), so it is assigned first. Try D1 = Morning → accepted.
- Forward checking removes Morning from D2 and D3's domains: D2 = {Afternoon, Night}, D3 = {Afternoon, Night}.
- D2 now has 2 options (tied with D3); assign D2 next. Try D2 = Afternoon → accepted.
- Forward checking removes Afternoon from D3 (shift-uniqueness) and also prunes any D3 value not strictly after Afternoon: D3 = {Night}.
- Only one legal value remains for D3: Night. Assign D3 = Night → all constraints satisfied, no domain becomes empty.

Final valid schedule: D1 → Morning, D2 → Afternoon, D3 → Night. Because forward checking never produced an empty domain along this branch, the search succeeds without ever needing to backtrack.

2 Answer:

The 5×5 grid is treated as a graph where each free cell is a node and an edge connects two cells that are orthogonally adjacent and not separated by an obstacle.

Problem formulation: States = free grid cells (row, col); Initial state = S = (0,0); Goal state = G = (4,4); Actions = move Up/Down/Left/Right into an in-bounds, non-obstacle cell; Path cost = number of moves (uniform cost of 1 per move); Heuristic = Manhattan distance h(n) = |row_n - row_G| + |col_n - col_G|, used only to report how close each expanded node is to the goal, since BFS itself does not need a heuristic to guarantee optimality.

Because every move costs exactly 1, BFS's guarantee of finding the path with the fewest edges is equivalent to finding the least-cost path, so no heuristic-guided search (like A*) is required for optimality here — Manhattan distance is reported at each expansion purely as extra diagnostic information.

Frontier behaviour: BFS keeps a FIFO queue. Each time a node is dequeued, its unvisited, non-obstacle neighbours (checked in the order Right, Down, Left, Up in this implementation) are enqueued and marked visited immediately, which prevents the same cell from being queued twice. The search terminates the moment the goal is dequeued; from there, the path is rebuilt by following stored parent pointers back to S.

Result: with the obstacle layout used in this submission, BFS reaches the goal after expanding 17 nodes. The reconstructed optimal path is (0,0) → (0,1) → (0,2) → (1,2) → (2,2) → (2,3) → (2,4) → (3,4) → (4,4), which is 8 moves, giving an optimal cost of 8. This matches the Manhattan distance lower bound of 8 computed at the start node, confirming the path found is shortest possible.

3 Answer:

a) Effect of the constraints on decision-making

Restricting movement to four orthogonal directions limits the branching factor at every cell, so the robot's route options are always small and easy to enumerate. Obstacles remove certain neighbours from consideration entirely and can force long detours. The fact that the robot can only sense cells adjacent to its current position means it cannot compute a full path to the goal in advance — it can only ever plan using what it currently knows and must be

ready to revise that plan. The extra cost for risky zones means the cheapest-looking route in terms of steps is not always the cheapest in terms of total cost, so the agent must weigh distance against risk rather than simply minimising step count. Together these constraints push the robot toward an incremental, cautious, cost-aware decision process rather than a single one-shot computation.

b) Solution approach: an online Uniform Cost Search agent

The approach implemented here interleaves sensing and planning rather than running UCS once over a known map:

- Maintain a knowledge map that starts containing only the robot's current cell; unseen cells are optimistically assumed to be free until proven otherwise.
- Before each move, sense the four adjacent cells and record whether each is free, an obstacle, or a risky zone, updating the knowledge map.
- Run Uniform Cost Search from the current position toward the goal using only the knowledge map gathered so far, where normal cells cost 1 and risky cells cost 1 + 2 = 3.
- Take only the first step of the resulting UCS plan, physically move there, and add that step's cost to the running total.
- Repeat sensing and re-planning from the new position until the goal cell is reached.

This differs from a single offline UCS run because the frontier expansion in every planning cycle is limited to what has actually been sensed plus optimistic assumptions about unexplored cells, so the robot's committed path can change if a later sensing step reveals a cheaper or blocked route than it originally assumed.

c) AI concepts demonstrated

Intelligent agent: the robot continuously perceives its environment through adjacent-cell sensing, maintains an internal knowledge map, and selects actions that reduce its expected cost to the goal — the defining behaviour of an intelligent agent.

Rationality: at each decision point the robot chooses the action that is expected to minimise total accumulated cost given everything it currently knows, which is precisely what a rational agent is expected to do even though its knowledge is incomplete.

Environment type: the environment is partially observable (only adjacent cells are visible), dynamic in the sense that the robot's understanding of it changes over time as new areas are sensed, sequential (each move affects which cells become visible next and what options remain), and initially unknown (the full obstacle/risk layout is not given up front).

d) Justification

An online Uniform Cost Search agent is well suited to this task because UCS natively supports the variable move costs created by risky zones, something BFS/DFS cannot optimise for since they treat every edge as equal. Running UCS repeatedly as new information is sensed lets the robot commit to safe, low-cost moves without requiring full knowledge of the building layout in advance, which matches the partially observable, initially unknown environment described in the problem. This combination gives the robot both cost-optimality with respect to its current knowledge and the flexibility to adapt as it uncovers more of the environment, satisfying the safety, cost-minimisation, and dynamic-adaptation requirements of the scenario.